Intro to Programming

Procedures and Functions

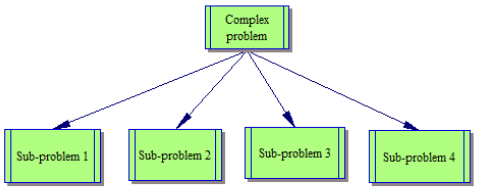
1

Introduction

- Programming is made easier when a complex programming problem is split into its smaller component parts.
- These smaller component parts are commonly called **subproblems** or **subprograms** or **subroutines**.

2

Decomposition



3

Types of subprograms

- There are two common types of subprograms:
  - Procedures
  - Functions
- Procedures and functions provide ways of making the program code:
  - more efficient
  - more easily understood.
  - more readable and
  - more maintainable.

4

Cont..

- When the initial programming problem is split down into smaller more easily solvable sub problems:
  - They can be coded as separate procedures or functions.
  - They can be tested individually before a program is complete.
    - This makes code easier to debug.
    - This also makes problems more manageable
    - These sub procedures and functions can be re-used
      - making your job of writing code easier.

5

Procedures/ Subroutine

- Is a block of code designed to complete a particular task.
- A section of a program that carries out some well defined operation on data specified by parameters.
- Can be called from anywhere in the program and different parameters can be provided for each call
- A call to a procedure interrupts the normal control sequence of a program.
- A procedure carries out a process without returning a value.

6

Cont..

```
PROCEDURE name (parameterlist)
    Statements
ENDPROCEDURE
```

7

Why use procedures/subroutines

- They can be called at any time using the subroutine's unique name.
- They allow you to gain an overview about how the program is put together.
- You can use a top-down approach to develop the whole project.
- The program is easier to test and debug because each subroutine is self-contained.
- Very large projects can be developed by more than one programmer.
- In terms of efficiency, they allow reuse of variable names and allocable memory space
- Can be reused reducing code size and executable file size.

8

Parameter and Argument

- A **Parameter** is a variable in the declaration of a function.
- A **parameter** is a variable in a method definition.
  - They identify values that are passed into a function.
- An **Argument** is the actual value of the variable that gets passed to function.

9

Function

- A program unit that, given values for input parameters, computes a value.
- Function is initiated by a call to it.
- A call to a function interrupts the normal control sequence of a program.
- When the instructions within the function have been completed, control is return to the instruction which called it.
- After which the next instruction is then executed.
- A function returns a value to the calling program.
- Repetitive functions only have to be written once.
- Complex functions can be called from within a subroutine, making the subroutine less complex.

10

Cont..

Data type FUNCTION name (parameterlist)
statements
return variable
ENDFUNCTION

11

Cont..

```
//Invoke (call) the method
int number1 = 25;
int number2 = 47;
int sum = add(number1, number2);

//Method definition
public int add(int x, int y)
{
   return (x + y);
}
```

actual parameters (or arguments)

formal parameters

12

Example function

```
INTEGER FUNCTION divide( a, b OF TYPE Integer)
    DISPLAY "Enter dividend"
    ACCEPT a
    DISPLAY "Enter devisor"
    ACCEPT b
      WHILE b=0
        DISPLAY "You cannot divide by zero"
        DISPLAY "Enter another number as divisor"
        ACCEPT b
      ENDWHILE
    answer = a/b
  return answer
ENDFUNCTION
```

13

Cont..

```
Main program
Use variables: answer, input1, input2 OF TYPE INTEGER
  DISPLAY "Enter first integer"
    ACCEPT input1
  DISPLAY "Enter second integer"
    ACCEPT input2
  answer = Add(input1,input2);
  PRINT answer
End program

INTEGER FUNCTION Add( x, y OF TYPE Integer)
        PRINT x
        PRINT y
      return x + y
ENDFUNCTION
```

14

Local and Global Variables

- **Local Variables**
- A Local variable only has scope within the subprogram where it has been defined.
- These variables only exist inside the specific function that creates them.
  - Are only known to the statements written in a function inside which they are declared.
  - They are not known to any other function present outside that function block.
- The values of local variables are lost/destroyed when a procedure or function call has concluded.

15

Cont..

- **Global Variables**
- Are declared outside all the functions present in a program.
- Can be accessed by any function comprising the program
  - they are known to each and every function present in a program
- Their value can be changed by any function present in the program.
- keep any values assigned to them
- Global variables remain in existence till the whole program get executed completely.

16

Cont..

- Global variables retain their values till the program is in execution phase.
  - they are stored at a fixed location decided by the compiler
- A global variable is destroyed when the entire program is terminated.

17

Cont..

- If a global and a local variable are declared with the same name then, all the statements of a code block in which local variable is declared will refer only to a local variable and will cause no effect to a global variable.

18

Cont..

```
// Global variables
int x;
int y;

INTEGER FUNCTION Add()
    return x + y
ENDFUNCTION

Main program
  Use variables: answer OF TYPE Integer // Local variable
    x= 5;
    y = 7;
    answer = Add()
  PRINT answer
End program
```

19

Passing parameters

- There are mainly two ways of passing parameters/arguments to function calls.
  - Passing by value (call by value)
  - Passing by reference (call by reference)
- Some languages only support call by value
  - E.g. C
  - the calling and called functions do not share any memory
    - they have their own copy
  - the called function cannot directly alter a variable in the calling function, it can only alter its private, temporary copy
- Some languages support both
  - E.g. C++, Java etc

20

Passing by value

- In call by value a copy of actual arguments is passed to respective formal arguments.
- Original value of variable is not modified.
- The changes made by the called function have no effect on the values of actual arguments in the calling function.

21

Cont..

```
VOID FUNCTION change(int num)
    PRINT "Before adding value inside function value of num is:" num
          num=num+100;
    PRINT "After adding value inside function value of num is:" num
ENDFUNCTION

Main program
    Use variables: x OF TYPE Integer
        x=100
    PRINT "Before function call value of x is:" x
      change(x) //passing value in function
    PRINT "After function call value of x is:" x
End program
```

Before function call value of x is:100
Before adding value inside function value of num is:100
After adding value inside function value of num is: 200
After function call value of x is:100

22

Passing by Reference

- Original value of the variable is modified.
- In call by reference the location (address) of actual arguments is passed to formal arguments.
- Therefore, any change made to formal arguments will also reflect in actual arguments.

23

Cont..

```
VOID FUNCTION change(int *num)
    PRINT "Before adding value inside function value of num is:" *num
          (*num) += 100
    PRINT "After adding value inside function value of num is:" *num
ENDFUNCTION

Main program
  Use variables: x OF TYPE Integer
        x=100
    PRINT "Before function call value of x is:" x
      change(&x) //passing reference in function
    PRINT "After function call value of x is:" x
End program
```

Before function call value of x is:100
Before adding value inside function value of num is:100
After adding value inside function value of num is: 200
After function call value of x is: 200

24

Cont..

call by value

- In *call by value*, a copy of actual arguments is passed to formal arguments of the called function
- Any change made to the formal arguments in the called function have no effect on the values of actual arguments in the calling function.

- In call by value, actual arguments will remain safe, they cannot be modified accidentally.

call by reference

- In *call by reference*, the location (address) of actual arguments is passed to formal arguments of the called function.
- This means by accessing the addresses of actual arguments we can alter them within from the called function.

- In *call by reference*, alteration to actual arguments is possible within from called function
- Therefore the code must handle arguments carefully else you get unexpected results.

25